

РЕКОМЕНДАЦИИ ПО ВЫПОЛНЕНИЮ ЭКСПЕРИМЕНТАЛЬНОГО ИССЛЕДОВАНИЯ

При выполнении экспериментального исследования в лабораторных работах можно контролировать наборы данных и задавать количества потоков перед вызовом цикла перебора функций.

В рамках исследования можно иметь массив указателей на функции функций или же контейнер (вектор) с функциями.

Рассмотрим организацию процесса экспериментального исследования на примере первой лабораторной работы. В рамках лабораторной работы воспользуемся организацией данных в массиве (векторе) Functions согласно заданию – следующим образом:

```
functions[0] = Заполнение одномерного массива (Последовательный)
functions[1] = Заполнение одномерного массива (Параллельный FOR)
functions[2] = Сложение двух одномерных массивов (Последовательный)
functions[3] = Сложение двух одномерных массивов (Параллельный FOR)
functions[4] = Сложение двух одномерных массивов (Параллельный Sections)
functions[5] = Подсчет суммы всех элементов итогового массива (Последовательный)
functions[6] = Подсчет суммы всех элементов итогового массива (Параллельный с
                использованием редукторов)
functions[7] = Подсчет суммы всех элементов итогового массива (Параллельный с
                использованием критических секций)
```

Для такого случая наиболее удобно будет хранить время в форме массива experiments_time[кол-во_НД][Кол-во_алгоритмов_функций][3]:

В случае статического выделения памяти будет массив:

```
double experiments_time[4][8][3]; // статический вариант объявления
```

В случае динамического выделения памяти понадобится следующий код для формирования массива:

```
int nd_count = 4; // количество наборов данных
int funct_cnt = 8; // количество исследуемых функций
double*** experiments_time = new double** [nd_count];
for (int nd = 0; nd < nd_count; nd++)
{
    experiments_time[nd] = new double*[ funct_cnt];
    for (int f = 0; f < funct_cnt; f++)
    {
        experiments_time[nd][f] = new double[3];
    }
}
```

При проведении экспериментального исследования необходимо организовать внешний цикл – перебор по наборам данных (в лабораторных работах обычно используются 4 набора данных).

В рамках первой лабораторной работы будут использованы следующие переменные для указания размера обрабатываемых массивов:

```
int nd_base_size = 100000; // Начальный размер набора данных
int nd_step = 70000;      // Шаг изменения набора данных
```

Сам же цикл обработки наборов данных примет форму:

```
//переменная nd отвечает за индекс набора данных
for (int nd = 0; nd < 4; nd++)
{
    int nd_size = nd_base_size + nd_step * nd;
    /*Внутри данного цикла будет вызов функции в которой будет
    происходить исследование набора данных*/
    test_functions(&experiments_time[nd][0], function_names, nd_size,
        iteration_cnt);
}
```

Функция test_functions будет отвечать за выделение памяти и проведение экспериментального исследования. Массив (вектор) функций также рекомендуется объявлять в данной функции. При этом рекомендуется организовать цикл, в рамках которого будет осуществляться изменение количества рабочих потоков

```
for (int threads = 1; threads <= 4; threads++)
{
    omp_set_num_threads(threads);
    //Вывод количества потоков на экран
    std::cout << "\n\n Количество потоков:" << omp_get_max_threads()
        << std::endl;
    //перебор алгоритмов по условиям и замер времени
    . . .
    //конец цикла перебора
}
```

Перебор алгоритмов в случае первой лабораторной работы можно осуществлять с учетом условия запуска. Так сначала сгруппировать запуск последовательных функций (алгоритмов), потом параллельных. Для этого можно написать условие запуска замера времени. В первой лабораторной работы функции с индексами 0 2 и 5 являются последовательными, остальные варианты представляют собой параллельные реализации. Принимая это во внимание код перебора функций и замера времени будет выглядеть следующим образом:

```
// Цикл перебора исследуемых функций
for (int f = 0; f < funct_cnt; f++)
{
    // Проверка количества потоков
    if (threads == 1)
    {
        // В случае только одного потока будут запускаться
```

```

// функции с индексами 0 2 5
if (f == 0 || f == 2 || f == 5)
{
    std::cout << function_names[f] << "\t";
    experiments_time[f][0] = MeasureFTime(***)/*Вызов функции
замера времени работы с учетом доверительного интервала */;
    experiments_time[f][1] = experiments_time[f][0];
    experiments_time[f][2] = experiments_time[f][0];
}
}
else
{
    // Если количество потоков отличается от одного то запускать
    // на исследования функции у которых индекс не равен 0 2 5
    if (f != 0 && f != 2 && f != 5)
    {
        std::cout << function_names[f] << "\t";
        experiments_time[f][threads - 2] = MeasureFTime(***)/*Вызов
функции замера времени работы с учетом доверительного интервала */
    }
}
}
}

```

Функция MeasureFTime(***) будет принимать разную форму в зависимости от того каким образом будет происходить работа.

Рассмотрим несколько случаев организации проведения экспериментального исследования – использование функциональных объектов STL и использование указателей на тестовые функции.

Использование функциональных объектов STL

В случае проведения экспериментов на основе функциональных объектов STL сформированных на основе лямбда выражений:

```

std::vector<std::function<double()>> Functions = {
    [&]() { return Fillarr_posl(mas1, mas2, size); },
    [&]() { return Fillarr_par_for(mas1, mas2, size); },
    [&]() { return SumArrays_posl(mas1, mas2, mas3, size); },
    [&]() { return SumArrays_par_for(mas1, mas2, mas3, size); },
    [&]() { return SumArrays_par_section(mas1, mas2, mas3, size); },
    [&]() { return ElArrSum_posl(mas3, size, tsum); },
    [&]() { return ElArrSum_for_reduction(mas3, size, tsum); },
    [&]() { return ElArrSum_for_crit_sec(mas3, size, tsum); }
};

```

Функция MeasureFTime(***) будет выглядеть следующим образом:

```

double MeasureFTime(std::function<double()> func, int expCNT)
{
    double* times = new double[expCNT];

```

```

for (int i = 0; i < expCNT; i++)
{
    times[i] = func();
    std::cout << "+";
}
std::cout << std::endl;
double avg_time = AvgTrustedIntervalMED(times, expCNT);
delete[] times;
return avg_time;
}

```

В таком случае вызов примет форму:

```
experiments_time[f][0] = MeasureFTime(Functions[f], iterations);
```

Использование указателей на функции

В случае использования указателей на функции, необходимо определить тип функции тестирования – для осуществления исследования, такие функции будут иметь общую структуру заголовка, но при этом будут вызывать соответствующие функции для исследования. Пример определения типа функции тестирования:

```

// Определение типа для функций тестирования
typedef double(*TestFuncTemp1)(double*, double*, double*, int);

```

Также необходимо написать функции для тестирования, согласно определенного типа, которые будут вызывать непосредственные реализации исследуемых функций. Пример функций тестирования для вызова последовательных функций:

```

/// Функции тестирования
double TestFillarr_pos1(double* arr1, double* arr2, double* empty2,
int size)
{
    return Fillarr_pos1(arr1, arr2, size);
}

double TestSumArrays_pos1(double* arr1, double* arr2, double*
arr_res, int size)
{
    return SumArrays_pos1(arr1, arr2, arr_res, size);
}

double TestElArrSum_pos1(double* empty1, double* empty2, double* arr,
int size)
{
    double tsum = 0;
    return ElArrSum_pos1(arr, size, tsum);
}

```

Создание массива содержащего указатели на тестовые функции будет выглядеть следующим образом:

```
void** Functions = new void* [8] {
    TestFillarr_posl,
    TestFillarr_par_for,
    TestSumArrays_posl,
    TestSumArrays_par_for,
    TestSumArrays_par_section,
    TestElArrSum_posl,
    TestElArrSum_for_reduction,
    TestElArrSum_for_crit_sec
};
```

Для такого случая функция MeasureFTime(***) будет выглядеть следующим образом:

```
double MeasureFTime(void* func, double* arr1, double* arr2,
                    double* arr3, int size, int expCNT)
{
    double* times = new double[expCNT];
    for (int i = 0; i < expCNT; i++)
    {
        times[i] = ((*TestFuncTempl)func)(arr1, arr2, arr3, size);
        std::cout << "+";
    }
    std::cout << std::endl;
    double avg_time = AvgTrustedIntervalMED(times, expCNT);
    delete[] times;
    return avg_time;
}
```

Код вызова замера времени в таком случае будет иметь следующий вид:

```
experiments_time[f][0] = MeasureFTime_ft(Functions[f], mas1, mas2,
    mas3, size, iterations);
```

Вывод результата исследования

Полученное в ходе экспериментального исследования время можно будет использовать для вычисления показателей необходимых для заполнения таблицы 1.2 или же просто вывести на экран и в текстовый файл.

Вывода данных на экран можно организовать следующим образом:

```
// Вывод на экран результатов экспериментального исследования
std::cout << "Результаты экспериментов:";
// Перебор по набору данных
for (int nd = 0; nd < 4; nd++)
{
    int nd_size = nd_base_size + nd_step * nd;
    // Вывод сведений о размере набора данных
```

```

std::cout << std::endl << "Размер НД:" << nd_size<<std::endl;
// Перебор функций для которых был произведен замер времени
for (int f = 0; f < funct_cnt; f++)
{
    // если это последовательная функция (индекс 0 2 или 5)
    if (f == 0 || f == 2 || f == 5)
    {
        // просто выводится имя функции и время
        std::cout << function_names[f] << "\t" <<
            experiments_time[nd][f][0] << std::endl;
    }
    else
    {
        /* в противном случае организуется цикл с выводом времени для
каждого из исследуемых потоков (2 3 4) */
        for (int th = 0; th < 3; th++)
            std::cout << function_names[f] << " [потоков:" << (th+2) <<
                "]\t" << experiments_time[nd][f][th] << std::endl;
    }
}
}

```